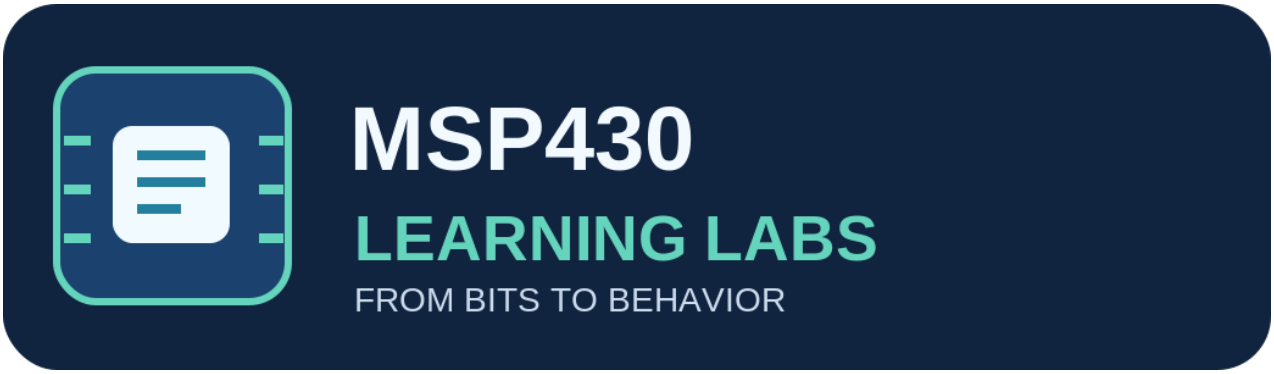


FROM BITS TO BEHAVIOR

An integrated MSP430G2553 curriculum

Eleven C laboratories • Four STEM sensor appendices • Edu430 firmware library • Thirty recovered assembly programs • PDP-11 comparisons

Instructor, student, and programmer pathway — 16 weeks / Version 1.0 / October 2026

Instructional question: How does an electrical signal become a program statement, an instruction, and a machine state—and how does that same idea appear in the PDP-11?

THE LADDER	THE EVIDENCE
Observation → EDU macro → C register → MSP430 assembly → PDP-11 comparison	Circuit output → register watch → instruction trace → measurement log → explanation

Original Joel Brigida assembly programs preserved unchanged under GPL-3.0. New curricular bridge code is additive; not all original assembly files have been target-built. The GitHub tree is the authoritative complete source archive.

01 Why one curriculum?

The collection previously contained multiple overlapping instructional artifacts: eleven C lessons, an expanding header-only Edu430 firmware library, STM32 VL6180X optical ToF over USCI_B0 I2C, other physics measurement appendices, a tiny scheduler, and thirty historical assembly examples. This integration turns those pieces into one learning trajectory while preserving their provenance and independence.

One coherent five-step experiment

STEP	STUDENT ACTION	INSTRUCTOR EVIDENCE
1 — Predict	Name the observable physical outcome.	Before-run prediction
2 — Express	Use a human-readable EDU_ command.	Annotated C source
3 — Reveal	Expand to real MSP430 register writes.	Register watch / pin map
4 — Trace	Single-step MSP430 instructions and flags.	PC, SP, Rn, flags trace
5 — Compare	Explain conceptual parallel or non-equivalence with PDP-11.	Architecture note and measured result

Preservation ≠ software certification

All 30 original .asm files were hash-verified against their upstream Git blob objects. This proves byte-for-byte recovery, not that those old samples compile under current CCS or are free of algorithmic errors. Source builds, simulator/host checks and physical hardware measurements must be recorded separately.

The original G2 and new G2ET LaunchPads share the MSP430G2553 silicon when that target is installed, but their USB debugger and board-level capabilities differ. The G2ET offers EnergyTrace instrumentation; the original requires additional measures for some modern CCS debugging configurations.

Course structure

The teaching spine is 16 weekly modules, designed for roughly three contact hours per week. Learners work on observable hardware in each module, interpret at least one register or instruction, and produce a short evidence artifact.

02 Sixteen-week scope and sequence

WK	TOPIC	EXISTING C LAB	ASSEMBLY / ARCHITECTURE FOCUS
01	Observe, power, and bit-level thinking	01-blink	SimpleVariables.asm
02	A machine built from registers	02-button	SampleInstructionSet.asm
03	Input, decisions and debouncing	03-patterns	Exercises/Lab2.asm
04	Arithmetic, flags and conditional branches	05-interrupt	Count-Binary-1s/CountBinary-1s.asm
05	Loops, shifts and algorithm equivalence	03-patterns	Multiplier/MultiplierShiftAdd.asm
06	Addresses, arrays and memory	10-telemetry	Array-Functions/FindMinArrayIndex.asm
07	Timers, interrupts and real time	04-timer	Flip-Bits-R5-R6/FlipBits-R5-R6.asm
08	PWM, clocks and power	06-pwm	SampleInstructionSet.asm
09	Stacks, calls and subroutines	08-uart-transmit	N-Factorial/CalcN-Factorial.asm
10	ADC and numerical measurement	07-adc	Exercises/MathInAssy-v1.asm
11	Serial protocol as scientific instrument	09-uart-commands	Sequence-Detection/SequenceDetection-Final.asm
12	SPI and I2C: reusable buses	08-uart-transmit	Exercises/Lab3-v1.asm
13	Optical time-of-flight exemplar	10-telemetry	Sequence-Detection/SequenceDetectionBytes.asm
14	Measurement science and calibration	10-telemetry	Array-Functions/AddTwoArrays.asm
15	Cooperative concurrency and low power	11-tiny-multitasker	Exercises/LabFinal.asm
16	PDP-11 comparative capstone	11-tiny-multitasker	SampleInstructionSet.asm

The complete lesson sheets are in `curriculum/weeks/`, with objectives, three conceptual passes, accessibility levels and exit tickets. Keep assembly as its own CCS startup project unless the lesson explicitly covers mixed-language integration.

03 Foundation: where each asset belongs

ASSET	PURPOSE	STUDENT ENTRY
Core C lessons 01–11	GPIO → interrupts → timing/PWM → ADC → UART → cooperative scheduler	One CCS main.c project per lesson
Original assembly archive	Preserve authentic examples for register/stack/addressing analysis	Standalone assembly projects; original untouched
Edu430 FwLib	Transparent macros for GPIO, clocks, timers, serial, SPI/I2C, ADC	Macro expansion and small inline functions
Sensor appendices A–D	Temperature, HX711, ultrasonic, VL6180X software I2C	Calibration and measurement quality
Optical hardware-I2C exemplar	VL6180X 16-bit register-address transactions, init, ranging	USCI_B0 bus capture and UART output
Bridge Labs 01 & 02	Deliberate C/ASM comparisons and corrected logic	Inspect flags and linker setup
PDP-11 reference materials	Historic 16-bit architectural comparison	Compare concepts, not machine code

Chip, board and software layers

```
App: edu_vl6180x_range_mm()           [millimeters and status]
Device: VL6180X register 0x0062        [16-bit reg address]
Bus: USCI_B0 repeated START            [I2C signals]
Board: P1.6 SCL, P1.7 SDA              [remove LED2 jumper]
Machine: USCI flags + clock + pin mux  [real hardware]
```

The same layered approach works for simple LED bit masks and the complex optical sensor. Use macros for elementary one-step operations, small functions for sequencing/timeouts and status values, and drivers for multi-register device protocols.

Block A — Visible bits and machine state

Digital I/O, pull-ups, flag effects and interrupts

Week 01 | Observe, power, and bit-level thinking

Observable / student action: Watch a GPIO bit change; inspect P1DIR/P1OUT. Hardware: LED and board identity.

Anchors: C lesson 01-blink; original assembly SimpleVariables.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 02 | A machine built from registers

Observable / student action: Trace MOV, BIS, BIC; compare PDP-11 register naming. Hardware: Button pull-up and LED.

Anchors: C lesson 02-button; original assembly SampleInstructionSet.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 03 | Input, decisions and debouncing

Observable / student action: Poll and debounce; test state transitions. Hardware: Button and two LEDs.

Anchors: C lesson 03-patterns; original assembly Exercises/Lab2.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 04 | Arithmetic, flags and conditional branches

Observable / student action: Build a flags truth table; compare C and assembly. Hardware: Single-step assembly.

Anchors: C lesson 05-interrupt; original assembly Count-Binary-1s/CountBinary-1s.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Block B — Algorithms, memory and time

Shift-add arithmetic, array addressing, timers, PWM and clock/power

Week 05 | Loops, shifts and algorithm equivalence

Observable / student action: Implement shift-add multiplication in C and ASM. Hardware: Debugger/register inspection.

Anchors: C lesson 03-patterns; original assembly Multiplier/MultiplierShiftAdd.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 06 | Addresses, arrays and memory

Observable / student action: Trace direct, indirect, indexed and autoincrement. Hardware: Memory browser.

Anchors: C lesson 10-telemetry; original assembly Array-Functions/FindMinArrayIndex.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 07 | Timers, interrupts and real time

Observable / student action: Measure timer interrupt timing; study status register. Hardware: LED, Timer_A.

Anchors: C lesson 04-timer; original assembly Flip-Bits-R5-R6/FlipBits-R5-R6.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 08 | PWM, clocks and power

Observable / student action: Duty cycle and EnergyTrace comparison. Hardware: LED, timer, optional G2ET.

Anchors: C lesson 06-pwm; original assembly SampleInstructionSet.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Block C — Connected peripherals

Stacks/subroutines, measurement, serial commands and reusable buses

Week 09 | Stacks, calls and subroutines

Observable / student action: Stack snapshots at CALL and RET; compare PDP-11. Hardware: Debugger, UART.

Anchors: C lesson 08-uart-transmit; original assembly N-Factorial/CalcN-Factorial.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 10 | ADC and numerical measurement

Observable / student action: Report ADC counts and volts with assumptions. Hardware: Potentiometer, TMP36 optional.

Anchors: C lesson 07-adc; original assembly Exercises/MathInAssy-v1.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 11 | Serial protocol as scientific instrument

Observable / student action: Design a command and data reporting grammar. Hardware: UART backchannel.

Anchors: C lesson 09-uart-commands; original assembly Sequence-Detection/SequenceDetection-Final.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 12 | SPI and I2C: reusable buses

Observable / student action: Identify START, address, ACK and repeated START. Hardware: I2C device / SPI flash optional.

Anchors: C lesson 08-uart-transmit; original assembly Exercises/Lab3-v1.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Block D — Inquiry and synthesis

Optical ToF, measurement calibration, concurrency and PDP-11 capstone

Week 13 | Optical time-of-flight exemplar

Observable / student action: Read VL6180X model ID, initialize and measure. Hardware: VL6180X on USCI_B0.

Anchors: C lesson 10-telemetry; original assembly Sequence-Detection/SequenceDetectionBytes.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 14 | Measurement science and calibration

Observable / student action: Compare scale, temp and ultrasonic uncertainty. Hardware: TMP36, HX711, HC-SR04.

Anchors: C lesson 10-telemetry; original assembly Array-Functions/AddTwoArrays.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 15 | Cooperative concurrency and low power

Observable / student action: Measure jitter, latency, EnergyTrace where available. Hardware: Three concurrent tasks.

Anchors: C lesson 11-tiny-multitasker; original assembly Exercises/LabFinal.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

Week 16 | PDP-11 comparative capstone

Observable / student action: Translate machine behaviors, present a documented instrument. Hardware: Capstone demonstration.

Anchors: C lesson 11-tiny-multitasker; original assembly SampleInstructionSet.asm. Each student submits a prediction, a code/register/flag explanation, and a measured or simulated observation.

04 Bridge Lab 01: one LED, three representations

Students perform a controlled equivalence test: a C bitwise instruction, its readable `EDU_` macro, and a TI assembly `XOR.B` all change the same observable LED output on P1.0. The exercise is *not* about memorizing assembler mnemonics; it is about tracing a physical state transition.

```
EDU_PIN_TOGGLE(P1,BIT0);  // Named student intent
P1OUT ^= BIT0;             // Equivalent C register write
xor.b #BIT0,&P1OUT        ; MSP430 assembly toggles the latch
```

Debugger protocol

Stop after setting P1DIR; write down the direction register. Then step over each GPIO toggle and record P1OUT bit 0. Explain how a write to a memory-mapped peripheral changes an electrical signal. Separate source-code time from actual observed time; the assembly example uses a simple delay loop, not a calibrated timer.

Code provided: `curriculum/bridge-labs/01-bits-and-registers/main.c` and `main.asm`. They are alternative programs and must not be compiled together because their startup symbols conflict.

PDP-11 comparison

A PDP-11 likewise supports direct manipulation of memory-mapped device words, but a historical PDP-11 memory address is not an MSP430 port register. Use identical conceptual verbs (move, set bits, clear bits), not identical board addresses. Both machines use a source,destination style, with different operand encodings.

05 Bridge Lab 02: bit counting as machine reasoning

The recovered original `CountBinary-1s.asm` uses `RLA.W` while its comments describe a rightward rotation. We preserve that historical artifact unchanged and supply a new transparent algorithm that counts all sixteen bits using `CLRC; RRC.W; JNC`.

```
C logic:       count += (word & 1u); word >>= 1;
MSP430 loop:    clrc
                rrc.w   R5       ; old bit 0 enters carry
                jnc     NOT_ONE
                inc.w   R6
NOT_ONE         dec.w   R7       ; exactly 16 iterations
                jnz     NEXT
```

With input 0xEDCB, the correct expected count is 11. Ask students to test 0x0000, 0xFFFF and 0x8000, then describe why arithmetic right shift and rotate-through-carry need deliberate treatment to implement an unsigned logical shift.

Inquiry extension

Measure instruction count under the debugger; compare a fixed-16-iteration loop with a C loop that terminates when the word becomes zero. Explain how a worst-case upper bound matters to the 1-ms cooperative scheduler.

Status

The standalone C bridge passes host syntax checks and the bit-counting reference test passes numerous representative 16-bit values. The assembly example still needs a genuine MSP430 assembler build and debug trace.

06 PDP-11 / MSP430 comparative architecture

Do not call the MSP430 a PDP-11 clone. They share substantial programmer-level ideas, but code and CPU registers do not correspond one-to-one.

TOPIC	MSP430G2553	PDP-11
Registers	16 registers: R0 PC, R1 SP, R2 SR/CG1, R3 CG2; R4–R15 general	Eight: R7 PC, R6 SP, R0–R5 general
Memory and I/O	16-bit address space, memory-mapped peripherals	Memory-mapped UNIBUS/Q-bus I/O (configuration-dependent)
Operands	Source then destination, byte/word forms	Source then destination, byte/word forms
Addressing	@Rn, @Rn+, X(Rn), immediate #n	(Rn), (Rn)+, X(Rn), immediate #n, broader deferred modes
Flags	N/Z/C/V in SR; compare as dst-src	N/Z/V/C in PSW; CMP as src-dst
Numeric convention	TI decimal numbers and explicit hex	MACRO-11 defaults to octal
Compatibility	Not binary compatible	Not binary compatible

Two key teaching traps: (1) MSP430 PC is R0 and PDP-11 PC is R7; (2) compare instructions may use different subtraction ordering for flags. Never infer branch equivalence solely from mnemonic similarity.

Representative register arithmetic

```
MSP430:  mov.w #3,R5   ; add.w #2,R5   ; bic.w #1,R5
PDP-11:  MOV   #3,R5   ; ADD   #2,R5   ; BIC   #1,R5
Result:  R5 = 4  (same algorithm; different machine code)
```

Primary sources: Texas Instruments *MSP430x2xx Family User’s Guide* (SLAU144) and Digital Equipment Corporation *PDP-11/40 Processor Handbook* and *PDP-11 Architecture Handbook*. Direct URLs and a longer treatment are in `curriculum/docs/PDP11_ARCHITECTURE.md`.

07 Scientific instruments, buses and measurement

A single reusable I2C reference

The ST VL6180X is the exemplar. The MSP430 USCI_B0 master exchanges address/data bytes; a device driver adds 16-bit register addresses, ST startup tuning, and conversion of register values to distance in millimeters. A bus NACK or timeout must be a reported result, not an infinite wait.

```
Read model ID:     I2C address 0x29, 16-bit register 0x0000
Read distance:     device register 0x0062 (ranging result)
Bus wires:         P1.6 SCL, P1.7 SDA; 3.3-V pull-ups
Other conflicts:   P1.6 LED2 must be disconnected;
                   USCI_B0 is SPI OR hardware I2C, not both.
```

The VL6180X is short-range; do not teach it as a long-range ultrasonic replacement. Check breakout VCC/VIN and logic-level support because the bare ST silicon is not natively a 3.3-V-only breakout.

The three other sensor pathways

APPENDIX	PHYSICAL QUESTION	METHODOLOGICAL CHECK
A — TMP36	How does voltage map to temperature?	Offset, slope, ADC resolution, noise
B — HX711	How does strain map to mass?	Tare, calibration masses, creep
C — HC-SR04	How does echo time map to distance?	Speed of sound, wall angle, safe 5-V ECHO division
D — VL6180X	How does optical ranging work?	Device identity, tuning, bus capture, 3.3-V safety

Every lab should log the known reference, multiple sensor readings, units, range, failure states and stated uncertainty. A graphic display without calibration is a demonstration, not a characterized measuring instrument.

08 Scheduling and energy as experimental quantities

Cooperative scheduling lets three small tasks share one MSP430G2553 without an RTOS or separate stacks. Lesson 11 progresses from blocking delays to explicit Timer_A logic, `RUN_EVERY` and optional LPM0 measurements.

Investigation protocol

Time LED blink, button sampling and UART reports with a 1-ms timer tick. Record missed button events under an intentionally blocking delay. Then use periodic jobs that return quickly. Compare average response latency and jitter under the same load. Use the G2ET EnergyTrace with identical test conditions when available.

LPM0 retains the SMCLK-driven Timer_A clock path in the reference lesson; LPM3 would need different clock choices. The timer interrupt interval is not proof that each task executes at zero latency or that all sensor drivers are nonblocking.

Risk register

RISK	HOW INSTRUCTOR CONTROLS IT
One task blocks others	Limit UART and I2C timeouts; inspect worst-case period
Timer tick counter wraps	Unsigned subtraction; bound maximum service delay
Device not responding	Explicit status / NACK / timeout and recovery
Supply or pin damage	Measure rails; 3.3-V GPIO; safe echo divider
Energy results incomparable	Same firmware workload, sampling interval and peripheral states

09 Assessment, learner access and credibility

Access route: follow known runnable code and annotate two register effects. Standard route: vary a physical parameter, predict and measure, explain one instruction or macro. Extension route: rewrite a state machine, validate a bus timing trace, or compare PDP-11 condition-code effects.

Suggested grading

EVIDENCE	WEIGHT
Weekly lab notebook and physical/debug trace	30%
Two architecture quizzes	15%
Sensor calibration report with uncertainty	15%
Bus and timing/scheduler investigation	15%
Team capstone and individual oral demonstration	25%

Verification ladder

Archive: source SHA matches upstream. Host syntax: mocked register header parses. Target build: genuine TI compiler/assembler/linker works. Hardware: behavior observed on an MSP430 board. Validated instrument: documented calibration and error bounds. Record these separately in the engineering notebook; do not merge them into one word such as “tested.”

Instructor safety checklist

- Confirm G2553 target and board revision before programming.
- Check reset-vector ownership: one C or assembly entry point only.
- Use 3.3 V at MSP430 GPIO; level-shift HC-SR04 ECHO.
- Disconnect P1.6 LED2 jumper for hardware I2C, and fit SDA/SCL pullups.
- Keep original upstream assembly unchanged; record proposed corrections separately.
- Do not claim PDP-11/MPS430 binary or assembly-source compatibility.

10 Publication, source recovery and next qualifications

The archival upstream repository contains 30 assembly source files in `Assembly-Lang`. The restored GitHub tree includes exactly those 30 file blobs; **all 30 hashes match the upstream versions**. Attribution: Joel Brigida / ADolbyB, GPL-3.0.

Restored sources:
https://github.com/RandoSY/nugget-physical-computing/tree/main/msp430g2553-learning-labs/historical/original-assembly/Assembly-Lang

Course spine: https://github.com/RandoSY/nugget-physical-computing/tree/main/msp430g2553-learning-labs/curriculum

Original upstream: https://github.com/ADolbyB/msp430g2553-labs/tree/main/Assembly-Lang

Known next steps

1. Compile representative historical assembly in current CCS, preserving originals and recording any nontrivial adaptation in a separate modern copy. 2. Run both new bridge `.asm` examples on a physical G2553. 3. Qualify USCI_B0 hardware I2C using VL6180X, with ACK/RESTART bus traces. 4. Reproduce sample instrument measurements with units and calibration. 5. Capture a documented comparison of PDP-11 and MSP430 flags using authoritative hardware manuals.

Local vs canonical distributions

This PDF is an instructor-facing synthesis. The local convenience ZIP includes current C laboratories, Edu430, new bridge labs, course materials and an upstream manifest. **The canonical GitHub branch ZIP includes the thirty restored assembly source files as well.** Always obtain the full GitHub repository ZIP when a fully self-contained assembly archive is required.

Authored October 2026 • Course integration v1.0 • Status: curriculum complete, historical source integrity verified, physical qualification pending.